Primeros pasos JUnit

1. Introducción

En esta guía vas a aprender desde cero a preparar un proyecto en IntelliJ con JUnit 5 y a escribir pruebas unitarias automatizadas. El objetivo es que, aunque no tengas experiencia previa con frameworks de testing, sepas crear un proyecto, configurar dependencias y convertir casos de prueba en tests que se ejecutan con un clic.

Trabajaremos con un ejemplo típico de clase: una función que calcula el total de un pedido aplicando reglas de descuento. A partir de esa función, diseñaremos casos de prueba y los automatizaremos con JUnit 5.

2. Maven e IntelliJ

2.1. Maven: por qué lo usamos aquí

Maven es un sistema de construcción y gestión de dependencias. En lugar de descargar librerías manualmente, Maven las descarga automáticamente y organiza el proyecto con una estructura estándar.

- pom.xml: archivo donde declaras dependencias (por ejemplo, JUnit) y plugins de construcción.
- src/main/java: código de la aplicación.
- src/test/java: código de pruebas.
- Fase 'test': Maven ejecuta las pruebas automáticamente (plugin Surefire).

2.2. IntelliJ: qué aporta al testing

IntelliJ detecta tus tests y te permite ejecutarlos con un clic. Te muestra resultados en verde (pasan) o rojo (fallan), te indica la línea exacta del fallo y, si lo deseas, puede medir cobertura (qué líneas o ramas del código se han ejecutado).

3. Caso de estudio: PrecioPedido

Vamos a probar un método que calcula el total de un pedido aplicando reglas de descuentos. Perfecto para practicar: validaciones, condiciones, límites y reglas que se pueden combinar.

3.1. Reglas de negocio

- base = precioUnitario × unidades
- Si unidades ≥ 10: descuento del 10%
- Si cliente = "VIP": descuento adicional del 5% (acumulable)
- Si cupón = "DESC20" y base ≥ 100: descuento del 20% (no acumulable; sustituye los anteriores)
- Si precioUnitario ≤ 0 o unidades ≤ 0: devuelve -1

3.2. Código a probar (PrecioPedido.java)

```
public class PrecioPedido {

    public static double calcularTotal(double precioUnitario, int
unidades, String cliente, String cupon) {

        if (precioUnitario <= 0 || unidades <= 0) {

            return -1;

        }

        double base = precioUnitario * unidades;

        double descuento = 0;


        if (unidades >= 10) {

            descuento += 0.10;

        }

        if ("VIP".equals(cliente)) {

            descuento += 0.05;

        }


        // Cupón especial: no acumulable

        if ("DESC20".equals(cupon) && base >= 100) {

            descuento = 0.20;

        }

        return base * (1 - descuento);

    }

}
```

3.3. Casos de prueba representativos

A continuación tienes un conjunto de casos que cubren: entradas inválidas, casos sin descuento, límites (9 vs 10), VIP, cupón especial y el caso de cupón presente pero no aplicable por base < 100.

ID	precioUnitario	unidades	cliente	cupón	esperado	nota
A1	0	5	NORMAL	—	-1	inválido: precioUnitario ≤ 0
A2	10	0	NORMAL	—	-1	inválido: unidades ≤ 0
A3	10	5	NORMAL	—	50.00	sin descuentos
A4	10	9	NORMAL	—	90.00	límite por debajo de 10
A5	10	10	NORMAL	—	90.00	límite en 10 (10%)
A6	20	2	VIP	—	38.00	VIP solo (5%)
A7	10	10	VIP	—	85.00	VIP + unidades (15%)
A8	10	10	VIP	DESC20	80.00	cupón anula (20%) con base=100
A9	9.99	10	NORMAL	DESC20	89.91	cupón no aplica (base<100), sí 10%
A10	12.4	8	VIP	DESC20	94.24	cupón no aplica (base<100), VIP sí

4. IntelliJ: configurar JUnit 5 y automatizar

4.1. Requisitos previos

- IntelliJ IDEA instalado
- Conexión a Internet para que Maven descargue dependencias la primera vez.

4.2. Crear la clase del código a probar (PrecioPedido)

1. Clic derecho en **src/main/java** -> New -> Java Class.
2. Nombre: PrecioPedido.
3. Pega el código del apartado 3.2 y guarda.

4.3. Añadir JUnit 5 al proyecto (pom.xml)

En Maven, 'instalar' JUnit significa declarar la dependencia en el pom.xml. La dependencia se marca con scope=test para indicar que solo se usa al ejecutar pruebas.

4. Abre pom.xml.
5. Dentro de <dependencies> añade junit-jupiter.
6. Añade el plugin maven-surefire-plugin para ejecutar tests (fase test).

Ejemplo mínimo recomendado (ajusta el JDK si tu proyecto usa otro):

```
<dependencies>
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter</artifactId>
        <version>5.10.2</version>
        <scope>test</scope>
    </dependency>
</dependencies>


<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-surefire-plugin</artifactId>
            <version>3.2.5</version>
        </plugin>
    </plugins>
</build>
```

Después de guardar, IntelliJ suele mostrar 'Load Maven Changes'. Pulsa para recargar. Si no aparece, abre la ventana Maven y pulsa 'Reload All Maven Projects'.

4.4. Crear la clase de test (PrecioPedidoTest)

Clic derecho en **src/test/java** -> New -> Java Class.
Nombre: PrecioPedidoTest.
Pega el código de test del apartado 4.5.

Importante: usa una organización coherente de packages. Si PrecioPedido no está en ningún package, PrecioPedidoTest tampoco. Si PrecioPedido está en un package, crea el test en el mismo package dentro de src/test/java (o impórtala correctamente).

4.5. Código de tests (JUnit 5)

Este test incluye dos pruebas para entradas inválidas (deben devolver -1) y un test parametrizado con varios casos. Así cubrimos particiones y límites sin duplicar mucho código.

Consejo: si al pegar el código aparecen textos corruptos (por ejemplo, "ort" en lugar de "import" o "ss" en lugar de "class"), vuelve a pegar desde una fuente limpia y revisa comillas y llaves.

```
import static org.junit.jupiter.api.Assertions.*;


import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;


class PrecioPedidoTest {


    // Para comparar decimales (double) usamos una tolerancia
    private static final double DELTA = 1e-6;


    @Test
    @DisplayName("Devuelve -1 si precioUnitario <= 0")
    void devuelveMenosUnoSiPrecioInvalido() {
        double total = PrecioPedido.calcularTotal(0, 5, "NORMAL",
null);
        assertEquals(-1, total, DELTA);
    }
```

```
    @Test
    @DisplayName("Devuelve -1 si unidades <= 0")
    void devuelveMenosUnoSiUnidadesInvalidas() {
        double total = PrecioPedido.calcularTotal(10, 0, "NORMAL",
null);
        assertEquals(-1, total, DELTA);
    }

    @ParameterizedTest(name = "[{index}] p={0}, u={1}, cliente={2},
cupon={3} => esperado={4}")
    @CsvSource(value = {
            "10, 5,  NORMAL, NULL,   50.0",
            "10, 9,  NORMAL, NULL,   90.0",
            "10, 10, NORMAL, NULL,   90.0",
            "20, 2,  VIP,    NULL,   38.0",
            "10, 10, VIP,    NULL,   85.0",
            "10, 10, VIP,    DESC20, 80.0",
            "9.99, 10, NORMAL, DESC20, 89.91",
            "12.4, 8, VIP, DESC20, 94.24"
    }, nullValues = "NULL")
    @DisplayName("Casos principales: particiones, límites y
combinaciones")
    void casosPrincipales(double precioUnitario, int unidades,
String cliente, String cupon, double esperado) {
        double total = PrecioPedido.calcularTotal(precioUnitario,
unidades, cliente, cupon);
        assertEquals(esperado, total, DELTA);
    }
}
```

4.6. Qué significa cada parte (explicación detallada)

- import static org.junit.jupiter.api.Assertions.*: permite usar assertEquals, assertTrue, etc. sin escribir el nombre completo.
- @Test: indica que el método es un test.
- @DisplayName: nombre legible que verás en el informe de tests (mejor que nombres técnicos).
- DELTA: margen para comparar doubles (por posibles diferencias muy pequeñas).

- @ParameterizedTest: un test que se repite con diferentes datos.
- @CsvSource: las filas de datos; cada fila alimenta una ejecución del test.
- nullValues = "NULL": permite escribir NULL en el CSV para que el parámetro sea null real.

4.7. Ejecutar los tests (y leer el resultado)

1. Abre PrecioPedidoTest.
2. Pulsa el triángulo verde al lado de la clase o del método y elige Run.
3. Si no ves el triángulo, ejecuta con Ctrl+Shift+F10 o con clic derecho dentro del archivo -> Run 'PrecioPedidoTest'.
4. Si tampoco aparece la opción Run, revisa que src/test/java esté marcado como Test Sources Root y recarga Maven (Reload All Maven Projects).
5. También puedes activar los marcadores desde File > Settings > Editor > General > Gutter Icons (Run line marker).
6. Observa el panel de resultados: verde significa que pasan; rojo indica que algún test falló.

Cuando un test falla, IntelliJ te muestra: qué test falló, qué valor se esperaba, qué valor se obtuvo, y la línea exacta del assert que falló.

4.8. Ejecutar con cobertura (Run with Coverage)

1. Clic derecho en PrecioPedidoTest y selecciona Run with Coverage.
2. Si no aparece la opción, abre Run > Edit Configurations..., comprueba que la configuración es de tipo JUnit y que apunta a la clase de test correcta.
3. También puedes lanzarlo desde el desplegable del botón Debug (Run with Coverage).
4. Observa en PrecioPedido.java qué líneas aparecen marcadas como ejecutadas o no ejecutadas.

Si una línea (o rama) no se ejecuta nunca, puede indicar que te falta un caso de prueba para ese camino.

5. Mini-regresión guiada: cambio 10% -> 12%

Ahora simulamos un cambio de requisitos: el descuento por unidades (unidades ≥ 10) pasa del 10% al 12%. Este ejercicio muestra por qué la automatización es tan útil: ejecutas todo en segundos y detectas qué comportamientos cambiaron.

5.1. Cambiar el código

7. En PrecioPedido.java, localiza el bloque: if (unidades >= 10) { descuento += 0.10; }
8. Cambia 0.10 por 0.12.
9. Guarda el archivo.

5.2. Ejecutar tests y entender fallos

Ejecuta PrecioPedidoTest otra vez. Algunos tests deberían fallar porque el comportamiento cambió (esto es correcto). Los fallos te dicen exactamente qué resultados esperados ya no coinciden.

5.3. Actualizar solo lo afectado

Actualiza únicamente los esperados que dependían del 10% (y donde no aplica el cupón del 20%):

- Caso 10,10,NORMAL,NULL: base=100; 12% -> 88.0
- Caso 10,10,VIP,NULL: descuento=12%+5%=17% -> 83.0
- Caso 9.99,10,NORMAL,DESC20: base=99.90; 12% -> 87.912

Vuelve a ejecutar. Si todo está actualizado correctamente, los tests vuelven a verde.

6. Problemas frecuentes (y cómo solucionarlos)

6.1. IntelliJ no encuentra los tests

- Asegúrate de que el test está en src/test/java y no en src/main/java.
- Revisa que el pom.xml contiene junit-jupiter con scope=test.
- Recarga el proyecto Maven (Load Maven Changes / Reload).
- Comprueba que el archivo termina en Test.java o que contiene métodos con @Test.
- Si ejecutas por configuración manual, usa una configuración JUnit de tipo Class apuntando a la clase de test.

6.2. Error de imports (no reconoce @Test o @ParameterizedTest)

- Normalmente se debe a que Maven no descargó las dependencias o no se recargó el proyecto.
- Revisa que el pom.xml se guardó y que no hay errores de sintaxis (etiquetas mal cerradas).
- Intenta 'Reload All Maven Projects' y después 'Rebuild Project'.
- Si persiste, prueba File > Invalidate Caches / Restart.

6.3. Fallos por decimales

Si comparas double sin DELTA, puede fallar por diferencias mínimas (por ejemplo, 87.9120000001). Usa assertEquals(esperado, obtenido, DELTA).

6.4. No aparece Run PrecioPedidoTest al hacer clic derecho

IntelliJ no detecta la clase como test si está fuera de src/test/java o si hay errores de compilación.

Busca la carpeta **src/test/java**.

Clic derecho sobre esa carpeta → **Mark Directory as** → **Test Sources Root**.

Recarga Maven y confirma que los métodos tienen anotaciones @Test/@ParameterizedTest.

6.5. Sale "No tests were found" o ClassNotFoundException

Suele ocurrir cuando la configuración apunta al nombre simple de la clase y no al nombre completo con package.

Revisa la configuración JUnit, corrige la clase objetivo y ejecuta Build > Rebuild Project.

6.6. No aparece "Run with Coverage"

Comprueba que la configuración sea JUnit y no Application/Maven.

Lánzalo desde el desplegable del botón Debug (Run with Coverage).

Si no aparece, configura "Run tests using" en IntelliJ IDEA y verifica que el plugin Coverage esté habilitado.